Чемпионат
по программированию

от Газпромбанка

Что нужно сделать

Разработать чат-ассистента для работы с базой данных университета.

Пользователь задаёт вопрос обычным языком. Ассистент составляет SQL-запрос, выполняет его в PostgreSQL и возвращает ответ с таблицей результатов.

Примеры запросов

Приёмная комиссия	Сколько заявлений подано на «Экономику» в 2026 году?
Преподаватель	Какой средний балл по моей дисциплине?
Ректорат	Как менялся набор студентов за последние пять лет?

Как должен работать ассистент

1	**Вопрос**	Пользователь вводит вопрос в чат.
2	**Генерация SQL**	Система получает схему БД и передаёт её языковой модели.
3	**Проверка**	Разрешены SELECT и таблицы из whitelist; применяются LIMIT и timeout.
4	**Выполнение**	Проверенный запрос выполняется в PostgreSQL.
5	**Ответ**	Пользователь видит ответ, SQL-запрос и таблицу результатов.

Если запрос завершился с ошибкой, система сообщает об этом и не придумывает ответ.

Что должно быть в прототипе

Работа с данными

- Подключение к PostgreSQL.
- Схема БД в промпте модели.
- Корректные JOIN и агрегатные функции.
- Ограничение количества строк через LIMIT.
- Обработка ошибок без вымышленных данных.

Интерфейс

- Поле ввода и кнопка отправки.
- История сообщений.
- Индикатор загрузки.
- Отображение SQL-запроса.
- Результат в виде таблицы.

Чат должен встраиваться в цифровые ресурсы университета как виджет.

Ограничения по запросам и данным

SQL-запросы

- Только SELECT.
- INSERT, UPDATE, DELETE, DROP и ALTER запрещены.
- Обращение только к разрешённым таблицам.
- Защита от SQL-инъекций.
- LIMIT и statement_timeout.

Персональные данные

Можно выводить ФИО преподавателей, деканов и сотрудников университета.

Данные студентов и абитуриентов — только в агрегированном или обезличенном виде.

Нельзя выводить сведения, позволяющие идентифицировать учащихся.

Паспортные, контактные и другие чувствительные данные не раскрываются.

Технические требования

Асинхронность	Запросы обрабатываются без блокировок.
Параллельная работа	Система обслуживает несколько пользователей одновременно.
Логирование	Фиксируются все действия пользователя.
Аналитика запросов	Предусмотрена возможность анализа запросов с помощью ИИ.
Встраиваемый виджет	Чат можно разместить на цифровых ресурсах университета.
Масштабируемость	Решение должно поддерживать рост нагрузки.

Для оценки производительности проверяются время ответа, объём выборки и работа под нагрузкой. Большие массивы данных в LLM не передаются.

Как будем оценивать

Основная часть — 100 баллов.

Критерий	Что оценивается	Баллы
Решение бизнес-кейса	Рабочий цикл, правильный SQL, польза для университета и защита данных	**30**
Техническая архитектура	Асинхронность, параллелизм, логи и тесты	**20**
Безопасность	SELECT, whitelist, защита от инъекций, timeout и персональные данные	**20**
Производительность	Скорость, объём выборки и работа под нагрузкой	**15**
Прогресс за день	Выполненная работа и промежуточные результаты	**8**
Командная работа	Распределение ролей и взаимодействие	**7**

Дополнительно — до 20 баллов. Максимальная оценка проекта — 120 баллов.

Дополнительные баллы

Большие выборки — до 10 баллов

Система оценивает объём выборки
и предлагает уточнить запрос.

- Автоматически добавляет LIMIT.
- Предупреждает об объёме выборки.
- Предлагает фильтры.
- Поддерживает пагинацию.
- Не передаёт 100 000+ строк в LLM.

Сам по себе LIMIT входит в обязательную часть.

Объяснение SQL — до 10 баллов

Отдельный блок Explainable AI
показывает, как устроен запрос:

- Какие таблицы используются.
- Какие JOIN и фильтры применены.
- Какие агрегаты вычисляются.
- Какие ограничения добавлены.

Без раскрытия внутренней
цепочки рассуждений модели.

Промежуточные проверки

Перед проверкой команда заполняет чек-лист.

На выступлении нужно:

- Кратко рассказать, что сделано.
- Показать работающие функции на экране.
- Назвать проблемы и план доработки.
- Ответить на вопросы экспертов.

10

минут
максимум

После выступления эксперты дают рекомендации по доработке проекта.

Финальная защита

Выступление — до 10 минут, с демонстрацией экрана.

Жюри задаёт 10 стандартных вопросов и проверяет,
как система справляется с задачами университета.

Проверяется:

- Правильность ответов и SQL-запросов.
- Соблюдение ограничений безопасности и персональных данных.
- Работа без перегрузки системы.
- Отсутствие вымышленных результатов.

Репозиторий проекта должен быть доступен для просмотра.

Перед началом работы

Проверьте на своих компьютерах

Python	`python3 --version`
Git	`git --version`
Docker	`docker --version`
Запуск контейнера	`docker run hello-world`

Пакеты должны устанавливаться через pip.

Проверьте интернет-соединение.

Подготовьте командную работу

- Доступ к личному GitHub.
- Настройки Git: user.name и user.email.
- Распределение ролей.
- Архитектуру решения.
- План разработки.

Что должно быть готово к защите

Рабочий прототип	Подключение к PostgreSQL, обработка вопросов, вывод SQL, ответа и таблицы.
Безопасность	Только SELECT, разрешённые таблицы, лимиты и защита персональных данных.
Демонстрация	Показ работы приложения на вопросах из университетских сценариев.
Репозиторий	Код проекта, доступный для просмотра жюри.

Побеждает команда, набравшая наибольшее количество баллов. Максимум — 120.